



DAFFODIL INTERNATIONAL UNIVERSITY

Department of Computing and Information System (CIS)

Algorithm

Semester Assignment

Submitted By:

Rashadul Islam

ID: 253-16-019

Section: 23(A)

Program: B.Sc. in CIS

Submitted To:

Mr. Israfil

**Lecturer, Department of Computing and Information System,
Faculty of Science and Information Technology, DIU**

Submission Date: 08-07-2026

Assignment Tasks

TASK-1

Q1. Explain why an efficient algorithm is necessary to organize emergency requests.

Answer: When a big emergency happens, lot's of people may ask for help at the same time. The system needs to store, priority, and search all these requests almost instantly.

If the system uses a slow method to handle this data, its work increases way too fast as more requests come in. The system fails to perform efficiently at the critical moment. For **SEHRS**, a slow method causes major problems such as Delay in identifying critical patients, System failure risk, Slow priority updates etc.

So, using a smart and fast algorithm keeps the system quick even when thousands of requests arrive at once. This ensures **fast response times, keeps the system running smoothly** and **saves lives**.

Q2. Which searching technique retrieves a record by Emergency ID efficiently, and what prerequisite must be met first?

Answer: The most efficient searching technique for finding a patient's record using a unique Emergency ID is Binary Search.

It repeatedly halves the search space by comparing the target Emergency ID with the middle element, giving $O(\log n)$ time — far faster than Linear Search's $O(n)$ once the number of stored requests reaches the thousands mentioned in the scenario.

Prerequisite:

Before applying Binary Search, all emergency records **must be sorted** according to the Emergency ID in ascending or descending order.

If the records are not sorted, Binary Search cannot work correctly.

Example:

Sorted Emergency IDs:

101	102	105	107	109	115
-----	-----	-----	-----	-----	-----

Suppose we need to search **109**.

- Check the middle element.
- If the middle value is smaller, search the right half.
- If the middle value is larger, search the left half.
- Continue until the record is found.

This makes Binary Search much faster than checking every record one by one.

TASK-2

Q3. Why does Bubble Sort become inefficient for a very large number of emergency requests, arranged by severity? Under what condition does it achieve its best time complexity?

Answer: Bubble Sort repeatedly compares two adjacent emergency requests and swaps them if they are in the wrong order.

As shown in Figure-01, the given pseudocode represents the Bubble Sort algorithm. To explain how this pseudocode works, let us assume an unsorted array of 4 element:

4	3	2	1
---	---	---	---

Here,

Length, $n=4$;

```
void bubbleSort(int arr[], int n)
{
    int i, j;

    for (i = 0; i < n - 1; i++)
    {
        for (j = 0; j < n - i - 1; j++)
        {
            if (arr[j] > arr[j + 1])
            {
                swap(&arr[j], &arr[j + 1]);
            }
        }
    }
}
```

Figure: 01

So, Outer loop runs $(n-1)$ times. It means Total passes = 3

In 1st pass, inner loop runs $(n-i-1)$ times. it means Comparison = $4-0-1=3$;

In 2nd pass, inner loop runs $(n-i-1)$ times. it means Comparison = $4-1-1=2$;

In 3rd pass, inner loop runs $(n-i-1)$ times. it means Comparison = $4-2-1=1$;

For an array of 4 elements, the bubble sort algorithm requires 3 passes and 6 comparisons to sort all element in ascending order. That's why, **when the number of emergency requests becomes very large**, Bubble Sort performs many comparisons and swaps, making it inefficient.

This gives:

- **Worst case / average case:** $O(n^2)$ comparisons and swaps
- **Best case** (already sorted, with an early-exit flag): $O(n)$

For thousands of emergency requests, $O(n^2)$ means that doubling the number of requests roughly quadruples the work. It becomes too slow for large datasets.

Bubble Sort achieves its **best time complexity $O(n)$ only when the list is already sorted** and an **optimized Bubble Sort** is used (with a swap flag). During the first pass, no swaps occur, so the algorithm stops immediately.

In conclusion, Bubble Sort is inefficient for a large number of emergency requests because it requires many comparisons and swaps. It performs best only when the emergency requests are already sorted.

Q4. Emergency requests are already sorted by severity level. Give the algorithmic steps or pseudocode to insert a newly received critical emergency while keeping the list sorted.

Answer: This is the single-insertion step used inside Insertion Sort. Assuming severity is stored so that the most severe request is first (descending order of severity):

```
InsertEmergency(requestList, newRequest, n){  
  
    i=n-1;  
  
    while(i>=0 && requestList[i] < newRequest)  
    {  
        requestList[i+1] = requestList[i];  
        i=i-1;  
    }  
  
    requestList[i+1] = newRequest;  
  
    n=n+1;  
  
}
```

TASK-3

Q5. Compare Linear Search and Binary Search.

Answer:

Here the Comparison between Linear Search and Binary Search:

<i>Feature</i>	Linear Search	Binary Search
<i>Searching Method</i>	Checks elements one by one	Repeatedly divides the search space into two halves
<i>Data Requirement</i>	Works on both sorted and unsorted data	Requires sorted data
<i>Best Case</i>	$O(1)$	$O(1)$
<i>Average Case</i>	$O(n)$	$O(\log n)$
<i>Worst Case</i>	$O(n)$	$O(\log n)$
<i>Performance</i>	Slower for large datasets	Faster for large datasets

Which is appropriate for finding a keyword inside the textual Emergency Description?

Answer:

Emergency Description contains text (strings) such as Road accident near DIU, Heart Attack, Fire Injury. These descriptions are usually not sorted alphabetically. Therefore, Binary Search cannot be applied.

Linear Search checks each emergency description one by one until the required keyword is found.

Hence, Linear Search is the most appropriate technique for searching a keyword inside the textual Emergency Description.

So, Binary Search for searching Emergency ID because IDs can be sorted and Linear Search for searching Emergency Description because the text is not sorted.

Q6. Analyze Bin Packing strategies for loading relief supplies (max 50 kg per ambulance) and justify the most appropriate approach.

Answer: Bin packing here means: given medical packages of varying weights, minimise the number of ambulances (bins of capacity 50 kg) needed to carry them all.

Strategy	How it works	Advantages	Disadvantages
First Fit (FF)	Packages are placed into the first ambulance with enough remaining space. If no ambulance has enough space, a new ambulance is used.	• Simple • Fast • Easy to implement	May use more ambulances than necessary.
Best Fit (BF)	Each package is placed into the ambulance that leaves the least remaining space.	• Better space utilization. • Usually uses fewer ambulances than First Fit.	Slightly slower because more comparisons are required.
Worst Fit (WF)	Each package is placed into the ambulance with the largest remaining space.	• Prevents one ambulance from filling too quickly.	Usually requires more ambulances and Less efficient.
First Fit Decreasing (FFD)	Sort packages in decreasing order. Apply the First Fit algorithm.	• Excellent space utilization. • Requires fewer ambulances. • Widely used in practical applications.	Sorting takes additional time.

Comparison

Strategy	Speed	Space Utilization	Ambulances Required
First Fit	Fast	Good	Moderate
Best Fit	Medium	Better	Few
Worst Fit	Fast	Poor	More
First Fit Decreasing	Medium	Excellent	Minimum

So, First Fit Decreasing (FFD) is the most appropriate strategy for this scenario. It uses the minimum number of ambulances, gives maximizes ambulance capacity, reduces transportation cost, provides efficient emergency resource allocation.

Q7. Perform a Depth-First Search (DFS) of the graph starting from Node A1.

Answer:

Depth-First Search (DFS) is a graph traversal algorithm that explores one path as deeply as possible before backtracking to explore other paths.

Here the graph with applying DFS algorithm—
Now, Start with A₁.

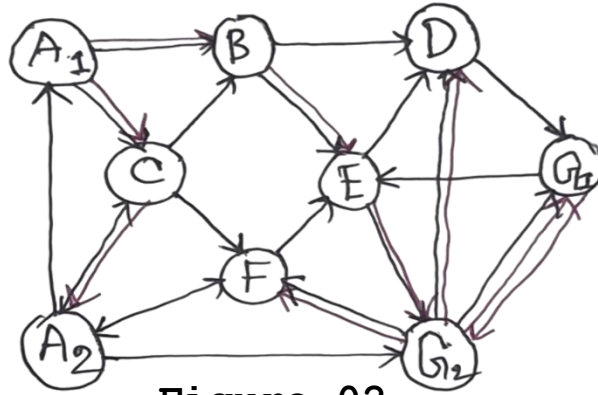
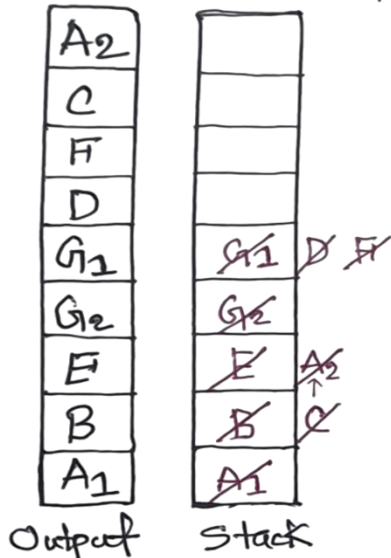
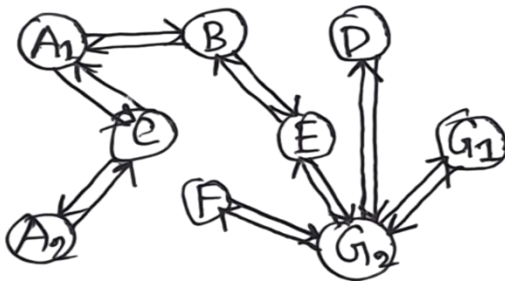


Figure-02

Here, the Final Graph—



The Final DFS Traversal = A₁ → B → E → G₂ → G₁ → D
→ F → C → A₂ .

TASK-4

Q8. Convert the graph to undirected form and find the MST using both Prim's and Kruskal's algorithms.

Answer:

Prim's Algorithm

Removing arrow directions but keeping weight gives the same 9 nodes and 18 undirected edges listed at the top of this assignment. Here the graph with constructing the MST - Starting from A_1 .

Step 1: $A_1 \rightarrow C$

Step 2: $C \rightarrow B$

Step 3: $B \rightarrow E$

Step 4: $E \rightarrow G_1$

Step 5: $E \rightarrow D$

Step 6: $E \rightarrow F$

Step 7: $F \rightarrow A_2$

Step 8: $D \rightarrow G_1$

Step 9: —

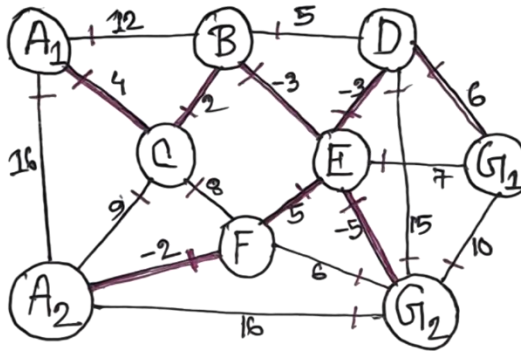
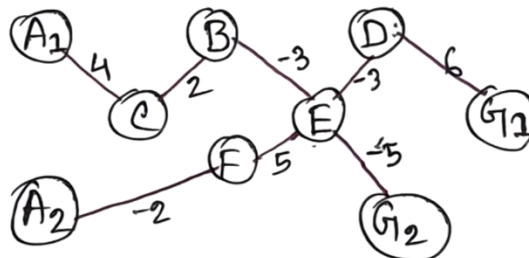


Figure-03

Here the Minimum Spanning Tree -



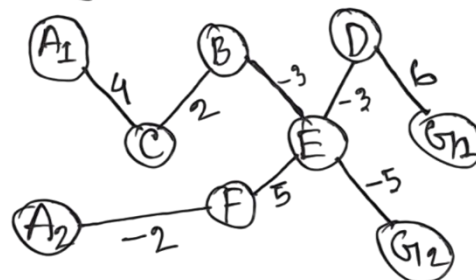
$$\therefore \text{Total Mst Cost} = 4 + 2 - 3 - 3 + 6 - 5 + 5 - 2 = 4$$

Kruskal's Algorithm

From the graph, sort the edges given above in the ascending order of their weights.

Edge	EE ₁	BE	ED	A ₁ F	CB	A ₁ E	BD	EF	DG ₁	FG ₂	GE	CF
Weight	-5	-3	-3	-2	2	4	5	5	6	6	7	8

Now, let's construct the Mst-



$$\therefore \text{Total Cost} = 4 + 2 - 3 - 3 + 6 - 5 + 5 - 2$$

$$= 4$$

Q9. Compare the MSTs from Prim's and Kruskal's — same tree or different, and why?

Answer:

Both Prim's and Kruskal's algorithms will always produce a Minimum Spanning Tree with the same total cost (4). This is a fundamental property of MSTs.

If all edge weights are unique, both algorithms will produce exactly the same tree structure. However, if some edges have equal weights, the structure of the resulting trees may differ slightly. Despite this difference, the total cost of the MST will always remain the same.

Additionally, the presence of negative weights does not affect the correctness of either algorithm, and both will still produce the optimal solution.

End
Thank You